

MySQL

Oracle-backed RDBMS
Lightweight Fast Reads

Thread per connection
(multiple execution path in single thread)

Ex : High-traffic e-commerce product catalog with many concurrent readers.

Default storage engine - InnoDB is ACID compliant

Concurrency Model | MVCC (InnoDB only)

PostgreSQL

Community-driven RDBMS
Advanced SQL Enterprise Ready Highly Extensible

Process per connection
(isolates workloads)

Ex : Banking app where one bad query should not destabilize other sessions.

Always ACID-compliant by design
financial or analytics systems — because of its MVCC, full ACID behavior , and rich SQL support.

Concurrency Model | MVCC (Multi-Version Concurrency Control)

Database Migrations Git for your database schema

Just like Git tracks every change you make to your code (and lets you undo them), migrations track every change you make to your database structure — and let you undo them too.

Two Functions:

- Up : Apply a change (Create a table, add a column)
- Down : Revert that change (Drop the table, remove the column)

Migration Log (tracked in a special table, e.g., "schema_migrations")

#	Migration Name	Status
1	001_create_users_table	Applied ✓
2	002_create_restaurants_table	Applied ✓
3	003_add_phone_to_users	Applied ✓

Changes made history

When New developer joins the team .His database is empty. He runs:

migrate up --> The system executes migrations in order --> now has the exact same database

Why Not Just Write SQL Manually?

Problem Without Migrations	How Migrations Solve It
"I forgot to add the phone column on staging"	Migrations run automatically; nothing is forgotten
"My DB doesn't match yours"	Everyone runs the same migration files in the same order
"Production broke after DB change"	Run migrate down to instantly roll back
"What changed in the DB last month?"	Read the migration files — they're version-controlled in Git
"New dev needs to set up DB from scratch"	One command: migrate up

Version Control Tools : Flyway ,Liquibase

Seeding --- Writing script (different migration file) -> putting some test data for testing purposes in dev env [dbmate new seed_data]

Type	Behavior
CHAR(n)	Fixed length — pads with spaces if shorter
VARCHAR(n)	Variable length — max n characters
TEXT	Variable length — unlimited (PostgreSQL-specific, no performance penalty vs VARCHAR)

Tip for Java devs: In PostgreSQL, TEXT and VARCHAR perform identically. Unlike Oracle/MySQL, there's no reason to avoid TEXT.

JSON stores as plain text — preserves formatting, slower to query.
JSONB stores in binary — discards whitespace/key order, but supports indexing and is much faster to query.

All constraint types: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, EXCLUDE

Table plus
dbgate